

```

1  //
=====
2  //  FILE: UIElementBase.cs
3  //  PATH: ./Engine/UI/Components/
4  //  MODULE: UI
5  //
6  //  ROLE:
7  //      Provide UI layout, interaction logic, or HUD rendering.
8  //
9  //  RESPONSIBILITIES:
10 //      - Provide Update() behavior for the UI subsystem.
11 //      - Provide HandleClick() behavior for the UI subsystem.
12 //      - Provide HandleMouseMove() behavior for the UI subsystem.
13 //      - Provide OnClicked() behavior for the UI subsystem.
14 //      - Provide OnMouseEnter() behavior for the UI subsystem.
15 //      - Provide OnMouseLeave() behavior for the UI subsystem.
16 //      - Provide OnMouseExit() behavior for the UI subsystem.
17 //      - Provide OnMousePress() behavior for the UI subsystem.
18 //      - Provide OnMouseRelease() behavior for the UI subsystem.
19 //      - Provide GetScreenBounds() behavior for the UI subsystem.
20 //      - Provide SetBounds() behavior for the UI subsystem.
21 //      - Provide CenterIn() behavior for the UI subsystem.
22 //      - Provide Show() behavior for the UI subsystem.
23 //      - Provide Hide() behavior for the UI subsystem.
24 //      - Provide ToggleVisibility() behavior for the UI subsystem.
25 //      - Provide GetEffectiveAlpha() behavior for the UI subsystem.
26 //
27 //  NON-RESPONSIBILITIES:
28 //      - Low-level data persistence or file serialization.
29 //
30 //  NOTES:
31 //      Auto-generated structure verified locally via file state scripts.
32 //
=====
33 /*
34 File:    UIElementBase.cs
35 Purpose: Base class for all UI components in the game.
36 Features: Common UI properties, positioning, visibility, event handling.
37
38 Created: February 14, 2026
39 Notes: Provides foundation for Button, Panel, HUD, and other UI elements.
40 */
41
42 using System;
43 using System.Drawing;
44 using SASZombieAssaultTD.Engine.Rendering;
45 using SASZombieAssaultTD.Engine.VectorMath;
46
47 namespace SASZombieAssaultTD.Engine.UI
48 {
49     /// <summary>
50     /// Represents different anchoring positions for UI elements.
51     /// </summary>
52     public enum UIAnchor
53     {
54         /// <summary>Anchor to top-left corner</summary>
55         TopLeft,
56
57         /// <summary>Anchor to top-right corner</summary>
58         TopRight,
59
60         /// <summary>Anchor to center of screen</summary>
61         Center,
62
63         /// <summary>Anchor to bottom-left corner</summary>
64         BottomLeft,
65

```

```

66     /// <summary>Anchor to bottom-right corner</summary>
67     BottomRight,
68
69     /// <summary>Anchor to top-center</summary>
70     TopCenter,
71
72     /// <summary>Anchor to bottom-center</summary>
73     BottomCenter,
74
75     /// <summary>Anchor to left-center</summary>
76     LeftCenter,
77
78     /// <summary>Anchor to right-center</summary>
79     RightCenter,
80
81     MiddleLeft,
82     MiddleCenter,
83     MiddleRight
84 }
85
86 /// <summary>
87 /// Base class for all UI elements in the game interface. Provides common
88 /// functionality for positioning, visibility,
89 /// and basic interaction. P11-04-09-E: Enhanced with anchoring, visibility toggles,
90 /// layering/z-index, and optional
91 /// fade-in/fade-out transitions. All properties are ECS-friendly.
92 /// </summary>
93 public abstract class UIElementBase
94 {
95     private bool _isVisible = true;
96     private bool _isEnabled = true;
97     private Core.Rectangle _bounds;
98     private float _alpha = 1.0f;
99     private float _targetAlpha = 1.0f;
100    private float _fadeSpeed = 2.0f;
101    private bool _isFading = false;
102    private bool _isMouseOver = false;
103
104    internal System.Numerics.Matrix3x2 Transform;
105    internal uint id;
106    internal object color;
107    internal object depth;
108
109    /// <summary>
110    /// Gets or sets the unique identifier for this UI element.
111    /// </summary>
112    public string Id { get; set; } = string.Empty;
113
114    /// <summary>
115    /// Gets or sets the position of the UI element.
116    /// </summary>
117    public Point Position
118    {
119        get => new Point((int)_bounds.X, (int)_bounds.Y);
120        set
121        {
122            _bounds = (Core.Rectangle)NewMethod(value);
123        }
124    }
125
126    private object NewMethod(Point value)
127    {
128        return Core.Rectangle.FromPositionAndSize(
129            (int)value.X,
130            (int)value.Y,
131            _bounds.Width,
132            _bounds.Height);
133    }

```

```

133     /// <summary>
134     /// Gets or sets the bounds rectangle of the UI element.
135     /// </summary>
136     public Core.Rectangle Bounds
137     {
138         get => _bounds;
139         set => _bounds = value;
140     }
141
142     /// <summary>
143     /// Gets or sets whether the UI element is visible.
144     /// </summary>
145     public bool IsVisible
146     {
147         get => _isVisible;
148         set => _isVisible = value;
149     }
150
151     /// <summary>
152     /// Gets or sets whether the UI element is enabled for interaction.
153     /// </summary>
154     public bool IsEnabled
155     {
156         get => _isEnabled;
157         set => _isEnabled = value;
158     }
159
160     /// <summary>
161     /// Gets or sets the parent UI element.
162     /// </summary>
163     public UIElementBase? Parent { get; set; }
164
165     /// <summary>
166     /// Gets or sets the z-order depth for rendering. Higher values render on top of
167     /// lower values. P11-04-09-E:
168     /// Enhanced layering support for UI elements.
169     /// </summary>
170     public int ZIndex { get; set; } = 0;
171
172     /// <summary>
173     /// Gets or sets the anchoring position for this UI element. P11-04-09-E:
174     /// Supports various anchoring positions
175     /// for automatic positioning.
176     /// </summary>
177     public UIAnchor Anchor { get; set; } = UIAnchor.TopLeft;
178
179     /// <summary>
180     /// Gets or sets the alpha transparency (0.0 to 1.0). P11-04-09-E: Supports
181     /// fade-in/fade-out transitions.
182     /// </summary>
183     public float Alpha
184     {
185         get => _alpha;
186         set => _targetAlpha = System.Math.Clamp(value, 0f, 1f);
187     }
188
189     /// <summary>
190     /// Gets or sets the fade transition speed in alpha units per second.
191     /// P11-04-09-E: Controls fade-in/fade-out
192     /// animation speed.
193     /// </summary>
194     public float FadeSpeed
195     {
196         get => _fadeSpeed;
197         set => _fadeSpeed = System.Math.Max(0.1f, value);
198     }
199
200     /// <summary>
201     /// Gets whether the element is currently fading. P11-04-09-E: Indicates if fade

```

```

198 transition is in progress.
199 /// </summary>
200 public bool IsFading => _isFading;
201
202 /// <summary>
203 /// Event triggered when the UI element is clicked.
204 /// </summary>
205 public event EventHandler<EventArgs>? Clicked;
206
207 /// <summary>
208 /// Event triggered when the mouse enters the UI element bounds.
209 /// </summary>
210 public event EventHandler<EventArgs>? MouseEnter;
211
212 /// <summary>
213 /// Event triggered when mouse leaves UI element bounds.
214 /// </summary>
215 public event EventHandler<EventArgs>? MouseLeave;
216
217 /// <summary>
218 /// Event triggered when mouse button is pressed on UI element.
219 /// </summary>
220 public event EventHandler<EventArgs>? MousePress;
221
222 /// <summary>
223 /// Event triggered when mouse button is released on UI element.
224 /// </summary>
225 public event EventHandler<EventArgs>? MouseRelease;
226
227 /// <summary>
228 /// Event triggered when mouse exits UI element bounds.
229 /// </summary>
230 public event EventHandler<EventArgs>? MouseExit;
231
232 /// <summary>
233 /// Initializes a new instance of the UIElementBase class.
234 /// </summary>
235 protected UIElementBase()
236 {
237     _bounds = new System.Drawing.Rectangle(
238         (int)(float)0,
239         (int)(float)0,
240         (int)(float)100,
241         (int)(float)50);
242 }
243
244 /// <summary>
245 /// Initializes a new instance of the UIElementBase class with specified bounds.
246 /// </summary>
247 /// <param name="bounds">The bounds rectangle for the UI element.</param>
248 protected UIElementBase(System.Drawing.Rectangle bounds)
249 {
250     _bounds = bounds;
251 }
252
253 /// <summary>
254 /// Renders the UI element. Must be implemented by derived classes.
255 /// </summary>
256 /// <param name="context">The rendering context.</param>
257 public abstract void Render(Rendering.IDrawingContext asRenderContext,
258                             IRenderContext context);
259
260 /// <summary>
261 /// Updates the UI element state. P11-04-09-E: Enhanced to handle fade
262 /// transitions and anchoring updates.
263 /// </summary>
264 /// <param name="deltaTime">Time elapsed since last update.</param>
265 public virtual void Update(float deltaTime)
266 {

```

```

264         UpdateFadeTransition(deltaTime);
265         UpdateAnchoring();
266     }
267
268     /// <summary>
269     /// Handles mouse click events.
270     /// </summary>
271     /// <param name="position">The mouse click position.</param>
272     /// <returns>True if the click was handled, false otherwise.</returns>
273     public virtual bool HandleClick(Point position)
274     {
275         if (!_isVisible || !_isEnabled)
276             return false;
277
278         if (_bounds.Contains(new Vector3(position.X, position.Y, 0f)))
279         {
280             OnClicked();
281             return true;
282         }
283
284         return false;
285     }
286
287     /// <summary>
288     /// Handles mouse move events.
289     /// </summary>
290     /// <param name="position">The current mouse position.</param>
291     /// <returns>True if the mouse is over this element, false otherwise.</returns>
292     public virtual bool HandleMouseMove(Point position)
293     {
294         if (!_isVisible)
295             return false;
296
297         bool isOver = _bounds.Contains(new Vector3(position.X, position.Y, 0f));
298
299         //Mouse enter/leave tracking
300         bool wasOver = _isMouseOver;
301         _isMouseOver = isOver;
302
303         if (isOver && !wasOver)
304         {
305             OnMouseEnter();
306         }
307         else if (!isOver && wasOver)
308         {
309             OnMouseLeave();
310         }
311
312         return isOver;
313     }
314
315     /// <summary>
316     /// Triggers the Clicked event.
317     /// </summary>
318     protected virtual void OnClicked()
319     {
320         Clicked?.Invoke(this, EventArgs.Empty);
321     }
322
323     /// <summary>
324     /// Triggers the MouseEnter event.
325     /// </summary>
326     protected virtual void OnMouseEnter()
327     {
328         MouseEnter?.Invoke(this, EventArgs.Empty);
329     }
330
331     /// <summary>
332     /// Triggers the MouseLeave event.

```

```

333     /// </summary>
334     protected virtual void OnMouseLeave()
335     {
336         MouseLeave?.Invoke(this, EventArgs.Empty);
337     }
338
339     /// <summary>
340     /// Triggers the MouseExit event.
341     /// </summary>
342     protected virtual void OnMouseExit()
343     {
344         MouseExit?.Invoke(this, EventArgs.Empty);
345     }
346
347     /// <summary>
348     /// Triggers the MousePress event.
349     /// </summary>
350     protected virtual void OnMousePress()
351     {
352         MousePress?.Invoke(this, EventArgs.Empty);
353     }
354
355     /// <summary>
356     /// Triggers the MouseRelease event.
357     /// </summary>
358     protected virtual void OnMouseRelease()
359     {
360         MouseRelease?.Invoke(this, EventArgs.Empty);
361     }
362
363     /// <summary>
364     /// Gets the screen-space bounds of this UI element, accounting for parent
365     /// transforms.
366     /// </summary>
367     /// <returns>The screen-space bounds rectangle.</returns>
368     public virtual System.Drawing.Rectangle GetScreenBounds()
369     {
370         if (Parent != null)
371         {
372             var parentBounds = Parent.GetScreenBounds();
373             return new System.Drawing.Rectangle(
374                 (int)(parentBounds.X + _bounds.X),
375                 (int)(parentBounds.Y + _bounds.Y),
376                 (int)(_bounds.Width),
377                 (int)(_bounds.Height));
378         }
379         return new System.Drawing.Rectangle(
380             (int)(_bounds.X),
381             (int)(_bounds.Y),
382             (int)(_bounds.Width),
383             (int)(_bounds.Height));
384     }
385
386     /// <summary>
387     /// Sets the bounds of the UI element.
388     /// </summary>
389     /// <param name="x">The x-coordinate.</param>
390     /// <param name="y">The y-coordinate.</param>
391     /// <param name="width">The width.</param>
392     /// <param name="height">The height.</param>
393     public void SetBounds(int x, int y, int width, int height)
394     {
395         _bounds = new System.Drawing.Rectangle(
396             (int)(float)x,
397             (int)(float)y,
398             (int)(float)width,
399             (int)(float)height);
400     }

```

```

401     }
402
403     /// <summary>
404     /// Centers the UI element within the specified bounds.
405     /// </summary>
406     /// <param name="containerBounds">The container bounds to center within.</param>
407     public void CenterIn(System.Drawing.Rectangle containerBounds)
408     {
409         float nx = containerBounds.X + (containerBounds.Width - _bounds.Width) / 2;
410         float ny = containerBounds.Y + (containerBounds.Height - _bounds.Height) / 2;
411         _bounds = new System.Drawing.Rectangle(
412             (int)nx,
413             (int)ny,
414             (int)_bounds.Width,
415             (int)_bounds.Height);
416     }
417
418     /// <summary>
419     /// Shows the UI element with optional fade-in effect. P11-04-09-E: Enhanced
420     visibility control with fade
421     /// transitions.
422     /// </summary>
423     /// <param name="fadeIn">Whether to fade in the element (default: true)</param>
424     public virtual void Show(bool fadeIn = true)
425     {
426         _isVisible = true;
427         if (fadeIn)
428         {
429             Alpha = 1.0f;
430         }
431         else
432         {
433             _alpha = 1.0f;
434             _targetAlpha = 1.0f;
435             _isFading = false;
436         }
437     }
438
439     /// <summary>
440     /// Hides the UI element with optional fade-out effect. P11-04-09-E: Enhanced
441     visibility control with fade
442     /// transitions.
443     /// </summary>
444     /// <param name="fadeOut">Whether to fade out the element (default: true)</param>
445     public virtual void Hide(bool fadeOut = true)
446     {
447         if (fadeOut)
448         {
449             Alpha = 0.0f;
450         }
451         else
452         {
453             _isVisible = false;
454             _alpha = 0.0f;
455             _targetAlpha = 0.0f;
456             _isFading = false;
457         }
458     }
459
460     /// <summary>
461     /// Toggles the visibility of the UI element. P11-04-09-E: Simple visibility
462     toggle functionality.
463     /// </summary>
464     public virtual void ToggleVisibility()
465     {
466         if (!_isVisible)
467         {
468             Hide();
469         }
470     }

```

```

467         else
468         {
469             Show();
470         }
471     }
472
473     /// <summary>
474     /// Updates fade transition for the UI element. P11-04-09-E: Handles smooth
475     /// fade-in/fade-out animations.
476     /// </summary>
477     private void UpdateFadeTransition(float deltaTime)
478     {
479         if (System.Math.Abs(_alpha - _targetAlpha) > 0.001f)
480         {
481             _isFading = true;
482             float fadeDirection = _targetAlpha > _alpha ? 1f : -1f;
483             _alpha += fadeDirection * _fadeSpeed * deltaTime;
484             _alpha = System.Math.Clamp(_alpha, 0f, 1f);
485
486             //Hide element when fully faded out
487             if (_alpha <= 0f && _targetAlpha <= 0f)
488             {
489                 _isVisible = false;
490             }
491             //Show element when starting to fade in
492             else if (_alpha > 0f && _targetAlpha > 0f && !_isVisible)
493             {
494                 _isVisible = true;
495             }
496         }
497         else
498         {
499             _isFading = false;
500         }
501     }
502
503     /// <summary>
504     /// Updates element position based on anchoring. P11-04-09-E: Handles automatic
505     /// positioning based on anchor
506     /// settings.
507     /// </summary>
508     private void UpdateAnchoring()
509     {
510         //This would typically use the screen dimensions from a render context
511         //For now, this is a placeholder that derived classes can override
512         //to implement specific anchoring behavior
513     }
514
515     /// <summary>
516     /// Gets the effective alpha value for rendering. P11-04-09-E: Combines element
517     /// alpha with parent alpha for
518     /// hierarchical transparency.
519     /// </summary>
520     protected float GetEffectiveAlpha()
521     {
522         if (Parent != null && Parent is UIElementBase parentElement)
523         {
524             return _alpha * parentElement.GetEffectiveAlpha();
525         }
526         return _alpha;
527     }
528
529     internal object GetSize()
530     {
531         throw new NotImplementedException();
532     }
533
534     internal void Render(Rendering.IDrawingContext asRenderContext, float deltaTime)
535     {

```



```
533         throw new NotImplementedException();
534     }
535 }
536 }
```